Lab 3: tmux, Chain of Pipes, and Graphviz

In this lab, we are learning the basics of tmux. We also create a chain of pipes to transform input via multiple steps to receive the desired output without having to write a program for it. Lastly, we're visualizing our dungeon map from Lab 1 using the Graphviz tool.

For each assignment, create a `git tag` when you have solved it and pushed to the repository. That way, it is easier to identify individual changes and create milestones which you can refer back to by the tag name. Find information about git tags with some examples online. Create **annotated tags** and share them (push to your repository). Since our changes are small in nature, the naming scheme for the tags are `lab-number.assignment-number`. Since this is lab 3 and we assume that you solved the first assignment in there, the tag is `3.1`. For another solved assignment, increase the version number (tag) to `3.2`.

Introduction to Tmux and Customization

Tmux is a terminal multiplexer that allows you to open multiple terminal sessions within a single window. It's highly useful on shared servers where you may want to keep sessions active, even if you log out temporarily. In this guide, you'll learn basic tmux commands, customize your tmux environment, and complete exercises to practice adding dynamic information to the status bar.

Basic Tmux Concepts

Key Components

1. Sessions: Each tmux session is an independent environment with its own windows and panes.
2. Windows: Inside a session, you can create multiple windows to organize tasks.
3. Panes: Each window can be split into multiple panes, allowing you to view and run commands side by side.

Basic Commands

To start a tmux session, enter:

```
tmux
```

Detaching and Reattaching

Tmux allows you to detach from a session and reattach to it later, a handy feature on a shared server.

- Detaching: Press `Ctrl + b`, then `d`.

- Reattaching: Reattach to a previous session with: `tmux attach-session`

Short forms for the above commands are `tmux at` or even `tmux a`.

List all sessions if needed with:

```
tmux list-sessions
```

A shorter version of this command is `tmux ls`.

Exiting Tmux

To close a session, type exit or press `Ctrl + d` in each window, or use:

```
tmux kill-session -t <session_name_or_number>
```

Configuring Tmux with .tmux.conf

You can customize tmux using a configuration file, `~/.tmux.conf`. Below are a few key configurations to get started.

1. Changing the tmux Activation Key

By default, tmux uses `Ctrl + b` as the activation (prefix) key. This is OK, but difficult to reach when pressing the Ctrl key with our small finger and reaching over to the middle of the keyboard where the b key is (on English and German keyboard layouts). You can change this in `~/.tmux.conf` (create it if it does not exist) to `Ctrl + s` (s for session) which is closer together:

```
set -g prefix C-s
unbind C-b          # Unbind the default prefix key
bind C-s send-prefix
```

2. Moving the Status Bar to the Top

To move the status bar to the top, add this line to `~/.tmux.conf`:

```
set-option -g status-position top
```

3. Customizing the Status Bar Appearance

Change the background color to blue and text color to white with:

```
set-option -g status-bg blue
set-option -g status-fg white
```

Pick colors that you like, you don't have to use the ones shown above.

4. Setting the Status Bar Update Interval

For dynamic information (like date and time), set how frequently it updates. For example:

```
set -g status-interval 15
```

Save the file and exit back to the shell. Reload your configuration with:

```
tmux source-file ~/.tmux.conf
```

Window Tiling in Tmux: Splitting Horizontally and Vertically

Tmux allows you to split a single window into multiple panes, facilitating simultaneous views of different tasks or commands. By tiling windows either horizontally or vertically, you can monitor multiple sessions or commands in real-time.

Default Tiling Commands

- **Horizontal Split:** Creates a new pane below the current pane: `Ctrl + b, "`
- **Vertical Split:** Creates a new pane to the right of the current pane: `Ctrl + b, %`

Redefining Keys for Tiling in .tmux.conf

To enhance the tmux layout experience, add the following key bindings to your `~/.tmux.conf` file, mapping the vertical and horizontal splits to `|` and `-`, respectively. Those resemble the type of split we want to perform.

Open `~/.tmux.conf` and add these configurations:

```
# Redefine split bindings
unbind '"'
unbind %

# Set | for vertical split (to the right of the current pane)
bind | split-window -h

# Set - for horizontal split (below the current pane)
bind - split-window -v
```

- `unbind` removes the default split bindings (`"` for horizontal and `%` for vertical), allowing us to replace them.
- `bind |` and `bind -` set the new commands:
  - `split-window -h` for a horizontal split (pane to the right).
  - `split-window -v` for a vertical split (pane below).

Apply Changes:

To reload the configuration after editing, execute: `tmux source-file ~/.tmux.conf` Commit your current `.tmux.conf` file into your repository. This may also be a good time to create and push a tag `3.1` for this milestone.

Now, within any tmux session, you can quickly create splits:

- Press `Ctrl + s`, followed by | for a vertical split.
- Press `Ctrl + s`, followed by − for a horizontal split.

Pane Navigation Commands

Once you have multiple panes, you'll likely want to navigate between them efficiently. Here are essential navigation commands:

- **Navigate in a Specific Direction:** Use arrow keys with `Ctrl + s` to move focus between adjacent panes.
- **Switch Panes:** `Ctrl + s`, then `o`

Assignment:

Split the window in two panes (either horizontally or vertically) and run `top` in one of them. Switch to the other pane, run a long running process like `sleep 3000` in it. You should see the sleep process appear in the pane with top in it.

Press `Ctrl + t` to see the remaining seconds the sleep command has left. Next, detach that session and log out of the server. Log back in and re-attach to the existing tmux session. Press `Ctrl + t` again to see that more seconds have passed in between logins. This demonstrates that detached tmux sessions continue running even though the user has logged out.

Assignment: Customizing the tmux Status Bar

Now, let's practice the Unix commands we've learned already by creating dynamic information displays in the tmux status bar. The status bar (now at the top because of our current configuration) is configurable to display user-defined information. Here is a simple example.

Change your `~/.tmux.conf` file to contain this:

```
set -g status-interval 10
set -g status-right "Date: #(date)"
```

Save, exit and reload tmux. The upper right corner contains the custom string `Date:` and the output of the date command, refreshed every 10 seconds. You can change the interval to decrease or increase the update interval.

Now it is your turn to add a customization instead of the date output.

Change the status bar so that it displays the CPU usage of our server as percentages (`top` in non-interactive mode (`-b`) or `vmstat` command). Filter the program output through a chain of pipes to get the needed information. Use `grep` and `cut` if necessary. Use their man pages to learn more about these tools.

Chain of Pipes

In this part of the lab, we want to create a chain of pipes on the shell that will echo the ten most frequently appearing words in a given text at the end. Pipe the file to others to format it in a way that makes it suitable for counting, and lastly present the results in descending order (ten most frequent words first) on the screen. An example output is:

```
12 the
10 and
7 or
```

Transform capital letters to lowercase before counting. Filter out special characters (sentence stops, exclamation marks, etc). Split the problem into logical steps: transforming/filtering, counting, and echoing the result.

`cat, head, sort, tr, uniq`

Read the man pages for these commands and look for options that will help you achieve the result you need. You may reuse commands in the chain of pipes. Use those programs, don't write your own to solve this assignment. Use the attached poem **Be glad your nose is on your face** by Jack Prelutsky as test input (separate file provided).

When you're done, pick another text of your choice and see if your chain of pipes works for that one, too.

To verify that the number of words found are accurate, use `grep -woi word` with a pipe to the `wc` program to count the number of occurrences of that word. That way, you have a separate, independent verification of your results. Check the man pages for `grep` and `wc` to learn about these parameters.

Commit your results to the repository. Create another tag to celebrate this milestone.

Graphviz

In the daily life of a programmer, sometimes there is a need for visualisation like program flows, dependencies, and other graph-like structures to make them more comprehensive. To do that without having to learn a complicated graphics program, we will use the command line to generate graphs. Such a program for creating graphs is GraphViz.

Graphviz reads a file with instructions on how nodes connect to each other and how they look like (shape, size, color, etc.). It then uses different layout engines to arrange the nodes and connections between them. Here are some good getting started introductions to Graphviz:

- Introduction to GraphViz.
- Sketchviz is a browser-based version.
- The dot Guide, which is one of the layout engines gets you a basic idea on how dot works.

Once you familiarized yourself with Graphviz, use it to create a graph of the dungeon in Lab 1. Make your dungeon map show which rooms connect to each other, not their contents. Use a different style for the portal connections. Don't get too fancy: the objective is to learn about Graphviz, so a simple graph is fine to begin with. Experiment with different layout engines and node shapes.

Create a PNG file for the resulting image of the map. Push this image to your Lab 3 directory in your GitLab repository. This results in the creation of the `3.3` tag as well.